

Software Requirements Specification

JOD tang (จอตัง) — a LINE chat assistant for student money tracking

Introduction to Software Engineering (15031001) · Milestone 2 — Requirements & Specification

Field	Value
Team name	Lumen Line
Project title	JOD tang — log money in one chat message, and know what is safe to spend
SDLC chosen in M1 + reason	Incremental — ship the chat-logging MVP first, then add budgets, then saving plans, then AI. Each increment is usable on its own, so a slipped increment never blocks the ones already delivered.
Date / version	v1.0 · 24 September 2026
GitHub repo	https://github.com/6931503045-design/Lumen_line
Running demo	https://lumen-line.onrender.com · screen index at <code>/prototype.html</code>

1. Introduction

1.1 Purpose and scope

University students run out of money before the end of the month without being able to say where it went. Two things cause this. First, logging an expense in a normal finance app takes four steps — open the app, pick a category, type the amount, save — so students stop logging within two or three weeks and the record is never complete enough to act on. Second, even a complete record only answers "where did my money go", which is a report about the past; it does not answer the question a student actually asks, which is "how much can I still spend today without breaking the month".

JOD tang answers exactly two questions: **"How much can I still spend today?"** and **"I want this thing — how much do I need to save each month to afford it?"** It removes the logging friction by living inside LINE, which students already have open: typing กาแฟ 80 ("coffee 80") in a chat records an 80-baht expense in about three seconds, and forwarded bank notification emails are parsed into transactions without the student doing anything at all. The MVP is five Must requirements. Success means a student reaches the end of the month knowing what they can spend, because the number was in front of them daily — not because they were disciplined about bookkeeping.

1.2 Users and stakeholders

Name / role	Kind	What they need	What they fear / forbid
University student, age 18–24, on a fixed monthly allowance	primary user	To log an expense in one message, and to see a single trustworthy number for what is still safe to spend today	Being told a confident number that turns out to be wrong; having to open a separate app to log a 30-baht snack
Course instructor / TA	secondary stakeholder	To trace every screen back to a numbered requirement and a test	A prototype that looks finished but cannot be opened or explained
LINE Messaging API	external supplier	Webhook signatures verified; push volume kept inside the free tier	Exceeding 280 push messages per month, which silently stops delivery
The student's bank (KBank / K PLUS)	external data source	Email notifications consumed read-only, with DKIM verified	Forged notification emails being accepted as real transactions

Google Gemini (AI provider)	external supplier	Bounded, non-financial use; personal data removed before any call	Personal or financial data leaving the system inside a prompt
-----------------------------	-------------------	---	---

2. Overall description

2.1 Product context — what it does / does not

JOD tang keeps one ledger per student and turns it into a daily spending figure and a saving plan. The table below is the whole scope for this term; anything not listed is out.

IN SCOPE (MVP this term)	OUT OF SCOPE (explicit promise)
• FR-1 Log an expense from a chat message • FR-2 Import bank notification emails • FR-3 Daily safe-to-spend figure • FR-4 Category budget with 80% / 100% alerts • FR-5 Saving plan with three options • Supporting features FR-6–FR-12 (see §3.7)	• Connecting to any bank API / Open Banking • Investment, stock or crypto tracking • Any currency other than Thai baht • Charging users; any payment or subscription • A native mobile app or a separate web app • AI computing any monetary figure or forecast • General-purpose chit-chat with the AI • Statistical forecasting (Monte Carlo, seasonality)

2.2 Assumptions and constraints

- **Assumption:** every user already has LINE installed and uses it daily; identity is the LINE account, so there is no separate sign-up.
- **Assumption:** the student can set up email forwarding from their bank app to the address the system gives them (a written guide is provided).
- **Constraint — money arithmetic:** all amounts are stored as positive integers in *satang* (1/100 baht). Floating-point arithmetic is forbidden anywhere on a monetary path.
- **Constraint — AI boundary:** AI may interpret language only. It must not compute, adjust or forecast any monetary figure, and it must not write to the database without an explicit human confirmation.
- **Constraint — AI must be optional:** with the AI provider disabled, every Must requirement still works.
- **Constraint — LINE free tier:** at most 280 push messages per user per month.
- **Constraint — data isolation:** the database has row-level security enabled with no policies, so every query must filter by the signed-in user's id at the application layer.
- **Constraint — schedule:** a 10-week build window shared with four other milestones (M1–M5); team of five.

3. Functional requirements

Format: *As a <role>, I want <goal>, so that <benefit>*. Every story traces back to a pain stated in §1.1, and every acceptance criterion is written Given–When–Then so that it can be executed as a test.

3.1 Priority summary (MoSCoW)

ID	Requirement	Priority	Why this priority
FR-1	Log an expense from a chat message	Must	Without it there is no data at all
FR-2	Import bank notification emails	Must	Covers the spending a student forgets to type
FR-3	Daily safe-to-spend figure	Must	This is question 1 — the product's reason to exist

FR-4	Category budget with 80% / 100% alerts	Must	Turns a passive number into a warning before the money is gone
FR-5	Saving plan with three options	Must	This is question 2
FR-6	Add / edit / delete a transaction on the web	Should	Corrections matter, but a wrong row can also be cancelled from chat
FR-7	Monthly summary and category breakdown chart	Should	Useful context; the safe-to-spend figure already carries the decision
FR-8	Transfer into a plan and track its progress	Should	A plan is still worth having before transfers are wired
FR-9	Recurring rules (allowance, rent, internet)	Should	Saves typing; the same rows can be entered by hand via FR-1
FR-10	Create, rename and delete categories	Should	Twelve sensible categories are seeded on first use
FR-11	Text commands in chat (summary, remaining, budget, plan)	Should	Same information is reachable from the web screens
FR-12	Push notifications with a monthly quota guard	Should	Alerts can ride along in a chat reply, which costs no quota
FR-13	AI interprets messages the parser cannot read	Could	By constraint the product must work without AI at all
FR-14	Read a transfer slip from a photo	Could	Depends on FR-13
FR-15	Simulate the impact of a purchase before buying	Could	Pleasant, not load-bearing

Five Must requirements, chosen so that the two questions in §1.1 can both be answered end to end. FR-6 to FR-15 are deliberately not Must: each one can be removed and the system still answers both questions.

3.2 FR-1 — Log an expense from a chat message [Must]

User story: As a student on a fixed allowance, I want to log a purchase by typing one short message in LINE, so that logging costs me so little effort that I actually keep doing it past week three.

Pain this traces to: logging in a normal finance app takes four steps, so students abandon it within two or three weeks and the record is never complete enough to trust.

1. **Given** a student sends the message กาแฟ 80, **when** the system parses it, **then** an expense of exactly 8000 satang is stored against that student with the category "Food", and the reply states the amount and category.
2. **Given** a student sends เงินเดือน 15000 ("salary 15000"), **when** the system parses it, **then** the row is stored as income, not as an expense.
3. **Given** a student sends ข้าว 1.2k, **when** the system parses it, **then** the amount is read as 1,200 baht (120000 satang).
4. **Wrong path. Given** a student sends a message with no number in it, **when** the system parses it, **then** nothing is written to the ledger and the reply shows an example of the accepted format.
5. **Wrong path. Given** a student sends กาแฟ -80, **when** the system parses it, **then** the message is rejected and no row is created, because stored amounts are positive integers only.
6. **Wrong path. Given** a student sends an amount above 10,000,000 baht, **when** the system parses it, **then** the message is rejected and the reply states the per-transaction ceiling.

3.3 FR-2 — Import bank notification emails [Must]

User story: As a student who pays by bank transfer, I want the notification emails my bank already sends me to become transactions by themselves, so that the spending I forget to type is still on the record.

Pain this traces to: the expenses a student forgets are exactly the ones that break the month, and a student who just paid is the least likely person to stop and type it in.

1. **Given** a K PLUS notification email for a 131.00-baht payment is forwarded to the student's ingest address, **when** the next poll runs, **then** an expense of 13100 satang is created for that student within five minutes.
2. **Given** the email reached the system through Gmail forwarding, **when** its signature is checked, **then** DKIM still verifies and the email is accepted.
3. **Wrong path. Given** an email whose DKIM signature does not verify, **when** it is processed, **then** it is skipped and no transaction is created, so that a forged notification cannot inject a row.
4. **Wrong path. Given** an email arriving at an address with no valid ingest token, **when** it is processed, **then** it is skipped, because the system cannot tell whose ledger it belongs to.
5. **Wrong path. Given** an email that has already been imported, **when** the same email is fetched again, **then** no duplicate transaction is created.

3.4 FR-3 — Daily safe-to-spend figure [Must]

User story: As a student deciding whether to buy something right now, I want one number telling me what I can still spend today, so that I can decide in two seconds instead of reading a report about last month.

Pain this traces to: a list of past transactions does not answer "can I afford this today"; the student has to do the arithmetic, which nobody does at the till.

Definition: remaining = income this month + scheduled income not yet posted – expenses this month – scheduled expenses not yet posted – the monthly saving amount of every active plan. Per-day = remaining ÷ days left in the month, counting today, in Asia/Bangkok. Computed fresh on every request; no AI involved.

1. **Given** a student has 3,000 baht remaining and 10 days left in the month, **when** they ask how much they can spend, **then** the answer is 300 baht per day.
2. **Given** the student has an active saving plan of 1,000 baht per month, **when** the figure is computed, **then** that 1,000 baht is subtracted before the division, so the plan is not silently spent.
3. **Given** today is the last day of the month, **when** the figure is computed, **then** the divisor is 1 and the calculation does not fail.
4. **Wrong path. Given** the student has already overspent and the remaining amount is negative, **when** the figure is shown, **then** the screen states how much they are over by and does not present a negative number as a daily allowance.
5. **Wrong path. Given** a student with fewer than seven days of history, **when** the figure is shown, **then** it carries a low-confidence label, because a number from three days of data is not yet trustworthy.

3.5 FR-4 — Category budget with 80% / 100% alerts [Must]

User story: As a student who overspends on food without noticing, I want to be warned while I still have money left, so that I can change what I do this week rather than discover the problem when the month is over.

Pain this traces to: overspending is discovered at the end of the month, when nothing can be done about it.

1. **Given** a 3,000-baht monthly food budget and 2,400 baht already spent, **when** the next food expense is recorded, **then** the student is warned that 80% of the budget is used, exactly once for that month.
2. **Given** the 80% warning has already been sent and spending reaches 3,000 baht, **when** the next expense is recorded, **then** a 100% warning is sent once and the 80% warning is not repeated.
3. **Given** the student changes the budget amount, **when** the new amount is saved, **then** both warnings become available again for that month.
4. **Wrong path. Given** spending stands at 79.99966% of the budget, **when** the threshold is evaluated, **then** no warning is sent, because a rounded percentage must not trigger the alert early and consume the month's single warning.
5. **Wrong path. Given** a request to set a budget of zero or a negative amount, **when** it is submitted, **then** it is rejected.
6. **Wrong path. Given** a request to set a budget on a category belonging to another student, **when** it is submitted, **then** it fails as not found, without revealing that the category exists.

3.6 FR-5 — Saving plan with three options [Must]

User story: As a student who wants to buy a 25,000-baht laptop, I want to be shown what saving for it would actually cost me per month, so that I can commit to a plan I can keep instead of guessing and abandoning it.

Pain this traces to: students set a savings goal by wishful thinking, miss it in month two, and stop saving entirely.

1. **Given** a student sets a target of 25,000 baht, **when** the system builds the plan, **then** it offers exactly three options, each stating a monthly amount, a finish month and a confidence level, all derived from that student's own recorded income and spending.
2. **Given** the three options are shown, **when** the student confirms one, **then** that plan becomes active and the other two are discarded.
3. **Wrong path.** **Given** the student has not confirmed any option, **when** the safe-to-spend figure is computed, **then** the unconfirmed plan is not subtracted, because nothing was agreed to.
4. **Wrong path.** **Given** the student already has three active plans, **when** they try to create a fourth, **then** it is refused with the reason stated.
5. **Wrong path.** **Given** the student's recorded spending leaves nothing to save each month, **when** a plan is requested, **then** the system says the target is not reachable yet instead of inventing a monthly figure.

3.7 Should and Could requirements

These are specified at story level for this milestone; full acceptance criteria are written when each one is scheduled.

ID	Priority	User story
FR-6	Should	As a student, I want to correct or delete a transaction the system read wrongly, so that the figures I rely on stay true.
FR-7	Should	As a student, I want to see this month's income, spending and a breakdown by category, so that I can see which habit is draining the month.
FR-8	Should	As a student saving for a goal, I want to move money into the plan and see progress, so that I know whether I am still on track.
FR-9	Should	As a student with fixed monthly costs, I want rent and allowance recorded automatically each cycle, so that I only type the irregular things.
FR-10	Should	As a student whose spending does not fit the default list, I want to create my own categories, so that the breakdown matches my life.
FR-11	Should	As a student already in the chat, I want to ask for a summary by typing a word, so that I do not have to open a screen for a one-line answer.
FR-12	Should	As a student, I want important warnings pushed to me, so that I hear about a problem without opening the app.
FR-13	Could	As a student, I want to type a sentence the way I would to a friend and still have it logged, so that I never have to learn a format.
FR-14	Could	As a student, I want to send a photo of a transfer slip and have it logged, so that I do not retype what is already on screen.
FR-15	Could	As a student about to buy something, I want to see what it does to the rest of my month first, so that I can decide with the consequence in front of me.

4. Non-functional requirements

Every statement below carries a number or a condition that a test can check. NFR-5 and NFR-6 cover security and personal data and are mandatory for this milestone.

ID	Kind	Statement (testable)	How we will check
NFR-1	performance	A chat message shall receive a reply within 3 seconds on university Wi-Fi.	Stopwatch from send to reply, 10 trials, on campus
NFR-2	performance	A forwarded bank email shall appear as a transaction within 5 minutes of arriving.	Send a real notification, compare its timestamp with the created row
NFR-3	accuracy	Every monetary value shall be a positive integer in satang; no floating-point value shall appear on a monetary path, giving 0 satang of rounding error.	Unit tests over the money utilities; code review of every service that touches an amount
NFR-4	reliability	The system shall send at most 280 push messages per user per month, reserving the last 20% for over-budget and plan alerts only.	Count rows in the push log for one user over a month; unit tests on the quota service
NFR-5	security (required)	A student shall be able to read and write only their own data; a request naming another student's record shall fail as not found. Zero endpoints may query without filtering on the signed-in user id.	Call an endpoint with another user's record id and confirm it is refused; review all 23 endpoints
NFR-6	privacy / PDPA (required)	Personal identifiers shall be removed from any text before it is sent to the AI provider, and no monetary balance shall be included in a prompt. The system shall store only what a transaction needs — amount, category, note, timestamp — and never chat history from other conversations.	Unit tests on the redaction step; inspect the exact payload sent to the provider
NFR-7	availability	With the AI provider switched off, all five Must requirements shall still work.	Set the AI flag to off with no API key present and run the full test suite
NFR-8	correctness	Given the same stored data, a figure shall be identical every time it is requested; no randomness may enter a monetary calculation.	Call the same endpoint 10 times and compare; grep the money engine for random sources
NFR-9	usability	A student who has never seen the system shall log their first expense in under 30 seconds without being shown instructions.	Timed trial with five students who have not used it

5. Use cases

5.1 Use-case list

Use case	Actor	Related FR	Brief success story
UC-1 Log expense by chat	Student	FR-1	Student types "coffee 80"; the expense is stored and confirmed in one reply.
UC-2 Import bank email	Bank → System	FR-2	A forwarded K PLUS email becomes a 131-baht expense within five minutes.
UC-3 Check safe-to-spend	Student	FR-3	Student opens the app and sees "300 baht per day" for the rest of the month.
UC-4 Set budget and be warned	Student ← Scheduler	FR-4	Student sets a 3,000-baht food budget and is warned on reaching 80%.
UC-5 Create a saving plan	Student	FR-5	Student enters a 25,000-baht target, is shown three options, and confirms one.

One use case per Must requirement. Supporting requirements FR-6 to FR-15 extend these five rather than introducing new actors.

5.2 Use-case diagram

Not drawn in this milestone. M2's task is naming the use cases correctly against the FR list above; the actor / association / <<include>> diagram belongs to M3 Modeling.

6. Out of scope

1. Connecting to any bank API or Open Banking service — outside what a student team can be authorised for.
2. Investment, stock or cryptocurrency tracking — does not serve either question in §1.1.
3. Any currency other than Thai baht.
4. Charging users; any payment, subscription or advertising feature.
5. A native mobile app or a standalone web app outside LINE.
6. AI computing, adjusting or forecasting any monetary figure.
7. General-purpose conversation with the AI beyond clarifying questions.
8. Statistical forecasting such as Monte Carlo simulation or seasonality modelling.

7. Traceability matrix (Golden Thread)

Every Must requirement traces backwards to a stated pain and forwards to an executable test. The test column names the file that checks it.

Problem / pain	Requirement	Solution / design	Feature built	Test
Logging takes four steps, so students quit within three weeks	FR-1 chat logging	Chat gateway → rule-based text parser → ledger write; no AI on this path	Type "coffee 80" in LINE and it is recorded	regexParser.test.ts, thaiNumber.test.ts
The spending students forget is what breaks the month	FR-2 email import	Mailbox poll every 5 minutes → DKIM check → bank-format parser → duplicate guard	Forwarded K PLUS emails appear as transactions	kbankParser.test.ts, emailDkim.test.ts, dedup.service.test.ts
"Where did it go" does not answer "can I afford this today"	FR-3 safe-to-spend	Money engine, integer satang only, divides remaining by days left in Bangkok time	Daily figure on the dashboard and via a chat command	safeToSpend.test.ts, thaiDate.test.ts
Overspending is discovered when it is too late to act	FR-4 budget alerts	Per-category monthly budget with database-level flags so each threshold fires once	Budget bar per category plus a warning in chat	budget.service.test.ts
Goals are set by wishful thinking and abandoned in month two	FR-5 saving plan	Capacity computed from the student's own 90-day history, with a safety ratio and an emergency buffer	Three plan options with confidence labels	plan.service.test.ts
A wrong number stated confidently is worse than no number	NFR-3, NFR-8	Money engine isolated from the AI module; integers throughout; no randomness	Every figure on screen comes from the engine	money.test.ts, summary.service.test.ts

Personal and financial data must not leak to a third party	NFR-5, NFR-6	Per-user filtering at the application layer; redaction before any AI call	Cross-user requests refused; prompts carry no identifiers	<code>session.test.ts</code> , <code>redact.test.ts</code>
--	-------------------------------	---	---	---

8. AI usage log

Every prompt that drafted a requirement or a paragraph in this document is recorded here, together with what we had to correct by hand.

Date	Tool / model	What we asked	What AI produced	What we changed / verified	What we learned
14 Sep 2026	Claude	Draft candidate user stories for a student expense-tracking chat bot.	Nine stories, most beginning "As a user, I want to track my expenses".	Cut to five. Replaced "user" with "student on a fixed allowance". Deleted three stories with no benefit clause, and two that were features we invented rather than pains we had observed.	AI drafts pad the list. Anything without a "so that" tied to a real pain is gold-plating.
17 Sep 2026	Claude	Write the acceptance criteria for the budget alert threshold.	Given–When–Then criteria that compared a rounded percentage against 80.	Rejected. Rounding makes 79.99966% become 80.0 and fires the alert early, burning the month's only warning. Rewrote the comparison using integer cross-multiplication and added criterion 4 in §3.5 as a regression test.	AI-written money logic reads correctly and is still wrong. Criteria have to name the boundary value explicitly.
18 Sep 2026	Claude	Draft non-functional requirements for the system.	"The system should be fast, secure and easy to use."	Rejected all three as unmeasurable. Replaced with NFR-1 (3 seconds), NFR-4 (280 messages), NFR-5 (cross-user request refused) and NFR-9 (30 seconds, five students).	Unless you supply the number yourself, AI writes the adjective.
22 Sep 2026	Claude	Why does a forwarded bank email fail to match a student?	Suggested the token was missing from the delivery header.	Verified against a real forwarded email and the suggestion was right for the wrong reason: Gmail strips the token from one header but preserves it in another. Fixed by reading the forwarding headers first, then confirmed with a real 131-baht transfer.	AI cannot know how an external service behaves. Only a real message settles it.
23 Sep 2026	Claude	Review this SRS against the course rubric.	Flagged that all eight requirements were marked Must.	Accepted. Re-sorted to five Must and ten Should/Could, and wrote the reason for each priority in §3.1.	"Everything is Must" means the prioritising was never done.

Course rule observed: AI may draft, but every requirement here has been verified by the team and each author can explain any FR without notes.

9. Self-check

Scorecard item	Score	Evidence
1. Scope (in / out) is clear and traces back to the M1 problem	2	§2.1 and §6
2. All actors that use the system are named	2	§1.2 — five roles including external suppliers
3. At least 5 FRs, numbered, written as user stories passing the 4 quality gates	2	§3 — 15 FRs, 5 detailed in full
4. At least 3 NFRs, each with a number	2	§4 — 9 NFRs, all measurable
5. Given–When–Then for every Must, including a wrong path	2	§3.2–§3.6 — 27 criteria, 14 of them wrong paths
6. MoSCoW applied, with 3–5 Must	2	§3.1 — exactly 5 Must, reason given per row
7. No solution or technology mixed into a requirement	2	Design detail is confined to §7 and the §3.4 definition note
8. Traceability: story ↔ FR ↔ acceptance	2	§7 — every Must row reaches a named test file
9. No gold-plating; every FR ties to a stated problem	2	"Pain this traces to" under each Must FR
10. Every FR passes the 4 quality gates (no vague words, no solution)	2	Named roles, "so that" clauses, numeric criteria throughout
Total	20 / 20	Target was 14/20

Course rule: you may use AI to draft. You must log it, verify it against a real stakeholder, and be able to explain every FR in the oral exam. Work you cannot explain earns no credit.